

Private GOC Ledger

Consistency Verification Without Decryption

Bachelor Thesis for the degree of Bachelor of Science (BSc)

University of Basel

Jakob Spiess

July 14, 2026

Abstract

This paper introduces a Private GOC Ledger, a ledger system that allows verification of consistency between encrypted entries without revealing their contents. Building on an existing ledger design, we extend it with non-interactive zero-knowledge proofs that enable equality checks over encrypted values. We describe the motivation, design, and implementation of these extensions and discuss the resulting guarantees and trade-offs.

Contents

1	Introduction	3
2	Relevant Literature	4
2.1	CRDT-based Infrastructure	5
2.2	Relevant Internal Logic	5
2.3	Illustrative Example	6
3	Full Design and Operation	6
3.1	Matrix Representation of Accounts and Ledgers	6
3.2	Explanation of encryption scheme	8
3.2.1	ElGamal Encryption	8
3.2.2	Additive ElGamal	11
3.2.3	Security	12
3.3	Motivation for Zero Knowledge Proof concerning equality of tuple entries	13
3.4	Implementation of this ZKP	13
3.5	Making the ZKP non-interactive using Fiat-Shamir Heuristics	15
4	Code Implementation	18
4.1	Programming Language and Design Rationale	18
4.2	High level design of the Software	18
4.3	Important Code Snippets	19
5	Evaluation	21
5.1	Implementation Setup	21
5.2	Runtime Characteristics	21
5.3	Storage Requirements	22
5.4	Scaling Considerations	22

6	Possible Extensions	22
6.1	Second Zero-Knowledge Proof for Balance Preservation	22
7	Conclusion	23
8	AI Disclaimer	23

1 Introduction

Distributed ledgers are traditionally implemented using consensus protocols that enforce a total order over all transactions. While such mechanisms effectively prevent double spending and ensure consistency across participants, they are often stronger than required for many practical applications. In particular, enforcing global agreement on every update introduces substantial performance overhead and increases system complexity.

Recent work has shown that a ledger can be constructed without global consensus by relying on state-based replication and conflict-free replicated data types (CRDTs). In this model, ledger replicas exchange and merge state rather than individual operations, and convergence is guaranteed by the algebraic properties of the replicated data structure. The grow-only counter (GOC) ledger presented in [1] demonstrates that such an approach can provide a simple and efficient alternative to traditional blockchain-style systems.

The GOC ledger represents the global ledger state as a matrix in which each entry stores the cumulative value transferred from one account to another. Because the stored values are monotonic, replicas can merge their local states by taking element-wise maxima, ensuring that all replicas eventually converge to the same ledger state.

However, this design assumes that all ledger state is stored in plaintext. As a consequence, any entity with access to a ledger replica, or even a sequence of published snapshots, can directly observe account balances, transaction volumes, and interaction patterns between participants. Over time, such observations can reveal detailed behavioral information about users, including payment relationships and transaction frequencies.

A natural approach to mitigating this problem is to encrypt the ledger state. However, introducing encryption fundamentally changes the design constraints of the system. In particular, the original representation of each matrix cell as a single aggregated value is not compatible with state-based merging when replicas cannot decrypt the stored values. If replicas only exchanged encrypted aggregate values, they would be unable to merge concurrent updates originating from different participants without decrypting the state, which would defeat the purpose of encryption.

To address this issue, we adopt a different representation of ledger state in which each matrix entry stores a set of encrypted transaction contributions rather than a single aggregated value. Each element of the set represents an individual transfer encoded as a pair of ciphertexts encrypted under the sender's and receiver's public keys. Because encrypted transactions are unique with overwhelming probability the set data structure is fitting, in other words it is for all practical purposes impossible to accidentally generated the same element twice which has to do with randomness used in the encryption algorithm. Replicas can merge ledger states by taking the union of these sets without decrypting their contents. This representation preserves the monotonic growth property required for deterministic state merging.

While this representation allows encrypted ledger states to be combined without decryption, it introduces a new challenge: participants must be able to verify the correctness of transactions without learning the underlying values. In particular, a receiver must be able to check that an incoming transfer is well-formed and that the sender is not spending more funds than they possess. These checks cannot rely on plaintext balances and therefore require cryptographic mechanisms that allow correctness properties to be verified without revealing sensitive information.

Zero-knowledge proofs provide a natural solution to this problem. With the help of an appropriate proof for the encrypted transactions, participants can verify that a transaction encodes a valid transfer and that it preserves the ledger's balance constraints while keeping the corresponding values hidden. So much for why a zero-knowledge proof is needed, what specifically gets proven is explained in detail later in the paper.

Motivation and Contributions

This work investigates how better privacy mechanisms can be integrated into the CRDT-based ledger architecture described above while preserving its ability to combine independently produced state updates.

The overall cryptographic direction of the system was developed in close collaboration with my supervisor, Osman Bicer. In particular, the use of a homomorphic ElGamal encryption scheme for representing transaction values was proposed by Osman, as it allows encrypted amounts to be combined algebraically without revealing their underlying plaintext values. Osman also designed the zero-knowledge proof used in this system and explained its construction so that it could be integrated and implemented as part of this work.

Beyond the choice of cryptographic primitives, an important design challenge was adapting the original CRDT-based ledger representation to operate on encrypted data. In the original grow-only counter ledger, each matrix entry stores an aggregated numeric value and replicas merge states by taking element-wise maxima. This approach relies fundamentally on the ability to compare stored values, which is not possible once transaction values are encrypted.

During the design process, we therefore jointly explored alternative representations that would allow replicas to combine encrypted state without requiring any information about the encrypted values themselves. Through a series of design discussions, we arrived at the representation used in this work, in which each matrix entry stores a set of encrypted transaction contributions rather than a single aggregated value. As mentioned, the encryption scheme produces randomized ciphertexts, even identical plaintext values result in distinct ciphertexts with overwhelming probability. As a result, each transaction contribution appears as a unique element in the set, and concurrent updates can be merged simply by taking the union of sets. This preserves the monotonic growth property required for deterministic state merging while avoiding any need to compare encrypted values.

Building on these ideas, the primary contributions of this thesis are the following:

- A detailed system design that integrates homomorphic encryption and zero-knowledge proofs with the CRDT-based ledger representation.
- The development of an encrypted transaction representation in which ledger entries consist of sets of encrypted contributions that remain compatible with deterministic state merging.
- The implementation of a prototype that realizes the encrypted ledger structure, transaction creation, and verification of the associated zero-knowledge proofs.
- An analysis of the remaining requirements for preventing double spending in this encrypted setting and an outline of how an additional zero-knowledge proof could enforce balance preservation.

Fully preventing double spending requires a further proof showing that a sender’s balance remains non-negative after a transfer. While we outline how such a proof could be constructed using homomorphic aggregation and range proof techniques, its complete design and integration are outside the scope of this work and remain an interesting direction for future research.

2 Relevant Literature

This section provides a high-level overview of the underlying system on which our work is based. We intentionally avoid restating formal definitions, algorithms, or proofs. All such technical details, including correctness arguments and convergence guarantees, can be found in the original GOC Ledger specification [1], which we consider the authoritative reference.

2.1 CRDT-based Infrastructure

The existing system implements a replicated ledger using a state-based CRDT architecture. Instead of keeping the ledger as a globally ordered sequence of transactions, the system models it as a collection of independently replicated account states. Each replica maintains its own local copy of the ledger and periodically exchanges state information with other replicas in the network.

The ledger is structured as a grow-only mapping from account identifiers to account states. Each account state represents an immutable snapshot that can be deterministically merged with other versions. When replicas communicate, they merge the corresponding account states using a least-upper-bound operation. This approach ensures convergence even under unreliable network conditions, such as message loss, duplication, or reordering.

A key design decision in this system is that replicas propagate state rather than individual operations. By exchanging state instead of transaction logs, the system significantly weakens communication assumptions and avoids the need to reason about globally consistent operation histories. As long as updated states are eventually disseminated transitively across all replicas, the system guarantees that all replicas converge to the same final ledger state.

2.2 Relevant Internal Logic

Each account is internally represented as a collection of monotonic counters that capture the cumulative effects of different types of operations. These counters include, for example, values that track token creation, token destruction, outgoing transfers, and acknowledged incoming transfers. All counters are grow-only, meaning that once a value has been increased, it is never decreased. This monotonicity is essential for ensuring that replicas can merge their states without conflicts.

Importantly, an account’s balance is not stored explicitly. Instead, it is computed as a derived value by summing all balance-increasing counters and subtracting all balance-decreasing counters. Because updates may occur concurrently at different replicas, this derived balance can temporarily become negative. However, the balance is always computed deterministically from the account state, and replicas eventually converge to the same result once all updates have been propagated.

Concurrency is handled through deterministic merge semantics. When two replicas hold different versions of the same account, the merge operation is performed by taking the maximum value of each corresponding counter. This guarantees that no update is lost and that the merge operation is associative, commutative, and idempotent. As a result, the system provides strong eventual consistency without requiring coordination between replicas.

In our implementation of the ledger, we adopt a simplified transfer model compared to the original design. In particular, we omit the explicit acceptance (acknowledgment) step on the receiver side. Whenever a transfer from one account to another is observed, the transfer is considered immediately accepted by the recipient. Consequently, the system no longer needs to distinguish between counters that track sent amounts and those that track accepted amounts. For any pair of accounts, the amount sent from one account to another is, by definition, equal to the amount received.

This simplification removes the need to maintain separate directional state, such as a $\text{Alice} \rightarrow \text{Bob}$ counter alongside a $\text{Bob} \leftarrow \text{Alice}$ counter. Instead, the transfer state between two accounts can be represented in a single shared structure. Conceptually, this reflects the assumption that a payment is always taken when it is received, and therefore the ledger does not need to encode conditional acceptance. As a result, the ledger state can be represented more compactly as a matrix of pairwise account interactions. Each matrix entry captures the cumulative transfer state between two accounts and can be interpreted symmetrically from both the sender’s and receiver’s perspectives. This representation improves clarity and reduces redundancy in the

stored state.

Under this model, the ledger can be viewed as a matrix whose rows and columns are indexed by account identifiers, and whose entries store the monotonic transfer counters shared between the corresponding accounts:

Sender \ Receiver	Alice	Bob	Carol	...
Alice	0	5	2	...
Bob	1	0	0	...
Carol	0	3	0	...
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

Each entry evolves monotonically under merge operations, thereby preserving the convergence guarantees of the underlying CRDT-based design while enabling a simple and more compact representation of transfer relationships.

The existing system provides a ledger with strong convergence guarantees under weak network assumptions. It tolerates replica crashes and recoveries, does not require reliable broadcast, and avoids global synchronization. These properties make it well-suited as a base layer for further extensions. Moreover, the system deliberately separates correctness guarantees from policy enforcement. While convergence and accounting safety are ensured, stricter constraints such as preventing negative balances or handling adversarial replicas are not enforced by default. This separation enables flexibility and performance, but also motivates the enhancements explored in later sections.

2.3 Illustrative Example

As an example, consider two replicas that hold the same account state with a positive balance. Each replica independently processes a valid outgoing transfer based on its local view. Since the updates occur concurrently, neither replica observes the other’s operation at the time it is executed.

When the replicas later exchange state, their account states are merged by taking the maximum of each internal counter. The resulting state reflects the combined effect of both transfers. The final balance is computed deterministically from the merged counters, without requiring rollback or conflict resolution.

This example illustrates the core design philosophy of the system: operations are locally validated, global consistency is achieved through state merging, and reconciliation is handled automatically by the structure of the data type rather than by coordination.

3 Full Design and Operation

This section provides a detailed explanation of our approach.

3.1 Matrix Representation of Accounts and Ledgers

As discussed in Section 2, the global state of a ledger can be represented, in unencrypted form, as a matrix

$$M \in (\mathbb{Z}_{\geq 0} \times \mathbb{Z}_{\geq 0})^{n \times n},$$

where each entry captures the cumulative transfer state between a pair of accounts.

In the encrypted setting, each matrix entry corresponds to the complete one-way transaction history between two accounts. Intuitively, an entry may be viewed either as the amount sent from one account to another or, equivalently, as the amount received by the counterparty.

Formally, for each pair of indices $(i, j) \in \{1, \dots, n\}^2$, the matrix entry M_{ij} is defined as the set

$$M_{ij} = \left\{ \left\langle C_{A_i}, C_{A_j} \right\rangle_k^{i,j} \mid k \in \mathbb{N}_0 \right\},$$

where each element represents a monotonic contribution to the transfer state between accounts A_i and A_j .

Here, C_{A_i} and C_{A_j} denote ciphertexts encrypting the same non-negative transaction value, encrypted once under the sender's public key and once under the receiver's public key, respectively. The index k serves as a logical identifier for elements in the transaction set of the cell and reflects monotonic growth of state rather than an ordered transaction sequence.

A natural question at this point is why each matrix cell stores a set of encrypted transactions rather than a single encrypted grow-only value, as in the original unencrypted design. While encrypted values can indeed be combined, as discussed later, this representation choice is driven by the requirements of merging ledger states that are themselves encrypted.

The goal is that ledger replicas may exchange state without the ability to decrypt individual entries. If each cell contained only an encrypted aggregate value, a replica receiving such a ledger would be unable to incorporate updates originating from other replicas, except potentially for entries corresponding to its own account and only after decryption. This would fundamentally break the state-based merge model. To its own account because these are the only matrix cells that a ledger holder could decrypt.

By representing each cell as a set of encrypted transaction contributions, merging two encrypted ledgers reduces to taking the union of corresponding cell sets. This operation can be performed entirely on ciphertexts and preserves the monotonic growth property required for convergence. No decryption is required during the merge, and all replicas can apply the same deterministic merge rule.

The uniqueness of elements within each set is ensured by the randomized nature of the encryption scheme: encrypting the same plaintext value multiple times produces distinct ciphertexts with overwhelming probability. As a result, each transaction contribution corresponds to a unique set element, and set union correctly captures the accumulation of concurrent updates.

To sum up, each matrix cell consists of a set of tuples encoding encrypted transaction contributions between two accounts. Each tuple represents a pair of non-negative integers, expressed in the smallest unit of currency (e.g., cents) for simplicity of accounting. The first component, C_{A_i} , is the transaction value encrypted under the public key of the sender associated with row i , while the second component, C_{A_j} , is the same value encrypted under the public key of the receiver associated with column j .

Thus, each pair $\langle C_{A_i}, C_{A_j} \rangle$ encodes the net value transferred from account A_i to account A_j in encrypted form, while remaining compatible with state-based merging and eventual consistency.

Here is a matrix representation of the just described system:

Sender \ Receiver	Alice	Bob	Carol	...
Alice	$\emptyset$	$\{a_1, a_2, \dots\}$	$\{c_1, c_2\}$	...
Bob	$\emptyset$	$\emptyset$	$\emptyset$	...
Carol	$\emptyset$	$\{b_1, b_2\}$	$\emptyset$	...
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

3.2 Explanation of encryption scheme

3.2.1 ElGamal Encryption

ElGamal encryption can be implemented over any cyclic group G . A common example is the multiplicative group of integers modulo n , where n takes the form $1, 2, 4, p^k$, or $2p^k$ for an odd prime p and integer $k > 0$.

The following paragraphs provide the necessary mathematical background: we define what a group is, explain cyclic groups and specifically the multiplicative group of integers modulo n , and describe the Discrete Logarithm Problem, which underpins the security of ElGamal encryption.

Group A group is a mathematical structure consisting of a set equipped with a binary operation that combines any two elements to produce another element within the same set. This operation must satisfy three key properties: it must be associative, there must exist an identity element, and every element must have an inverse. For instance, the integers under addition form a group.

Cyclic Group A cyclic group is a special type of group that can be generated from a single element. Formally, it is a set of invertible elements under an associative operation, containing an element g such that every other element of the group can be expressed as repeated applications of the operation to g or its inverse.

Elements in a cyclic group can be represented as:

- powers of g when using multiplicative notation, or
- multiples of g in additive notation.

The element g is referred to as a generator of the group.

More generally, any element g of a group generates a cyclic subgroup consisting of all its integer powers:

$$\langle g \rangle = \{g^k \mid k \in \mathbb{Z}\},$$

known as the cyclic subgroup generated by g . If g is chosen as a generator of the whole group, then this subgroup coincides with the entire group.

The order of an element g , denoted $|g|$, is the number of distinct elements in the subgroup $\langle g \rangle$.

A cyclic group is a group that is equal to the cyclic subgroup generated by one of its elements:

$$G = \langle g \rangle$$

for some generator g .

Constructing a Group As a concrete example, we consider a multiplicative group modulo a prime number.

The group constructed in this section serves as the mathematical foundation for the encryption scheme and would be hardcoded in any implementation. The specific parameters chosen here are not suitable for real-world use, as their small size offers no meaningful security. However, this choice greatly simplifies computations and makes the underlying ideas easier to illustrate.

Recall that ElGamal encryption can be defined over any cyclic group G . In particular, the multiplicative group of integers modulo n is cyclic if and only if

$$n \in \{1, 2, 4, p^k, 2p^k\},$$

where p is an odd prime and $k > 0$.

We therefore choose p to be an odd prime, for example $p = 23$, and set $k = 1$. This yields $n = 23$, which satisfies the above condition. The multiplicative group of integers modulo 23 is denoted by

$$G = \mathbb{Z}_{23}^*.$$

Before describing this group explicitly, we recall why not all residues modulo n can be included. In a multiplicative group, every element must have a multiplicative inverse. An integer a has an inverse modulo n if and only if $\gcd(a, n) = 1$. Consequently, only those residues that are coprime to n are invertible modulo n and can therefore be elements of the group.

For this reason, the multiplicative group modulo n is defined as

$$\mathbb{Z}_n^* = \{[a] \mid \gcd(a, n) = 1\}.$$

In our case, $n = 23$ is prime. The only divisors of 23 are 1 and 23, so every integer a with $1 \leq a \leq 22$ satisfies $\gcd(a, 23) = 1$. Thus, every nonzero residue modulo 23 is invertible, and we obtain

$$\mathbb{Z}_{23}^* = \{1, 2, 3, \dots, 22\}.$$

Since $\mathbb{Z}_{23}^*$ is cyclic, there exists a generator $g \in \mathbb{Z}_{23}^*$ such that

$$\mathbb{Z}_{23}^* = \langle g \rangle.$$

I think there is no known algorithm to find a generator, but check this idk. For example, $g = 5$ is a generator modulo 23. This can be verified by computing successive powers of 5 modulo 23:

$$5^1 \bmod 23 = 5, \quad 5^2 \bmod 23 = 2, \quad 5^3 \bmod 23 = 10, \quad \dots$$

Continuing this process shows that every element of $\mathbb{Z}_{23}^*$ appears exactly once. The reader may verify the remaining powers to confirm this.

We therefore conclude that

$$\langle 5 \rangle = \{5^k \bmod 23 \mid k \in \mathbb{Z}\} = \mathbb{Z}_{23}^*,$$

that is, every element of the group can be written as a power of 5 modulo 23.

For comparison, consider the case $n = 15$, which is not prime. Although the residues modulo 15 are $\{0, 1, \dots, 14\}$, only those that are coprime to 15 are invertible. Hence,

$$\mathbb{Z}_{15}^* = \{1, 2, 4, 7, 8, 11, 13, 14\}.$$

If n is prime, every nonzero residue modulo n is invertible, resulting in a particularly simple group structure that is often cyclic. If n is composite, only a subset of residues is invertible, and the resulting group structure becomes more complicated.

ElGamal encryption is performed in three phases: the key generation, the encryption, and the decryption.

Key generation The first party, Alice, generates a key pair as follows:

- Generate an efficient description of a cyclic group G of order q with generator g . Let e represent the identity element of G .
It is not necessary to come up with a group and generator for each new key. Indeed, one may expect a specific implementation of ElGamal to be hardcoded to use a specific group, or a group from a specific suite. The choice of group is mostly about how large keys you want to use. We will explain our choice of group later in the implementation.
- Chose an integer x randomly from $\{1, \dots, q-1\}$
- Compute $h := g^x$
- The public key consists of the values (G, q, g, h) . Alice publishes this public key and retains x as her private key, which must be kept secret.

Encryption A second party, Bob, encrypts a message M to Alice under her public key (G, q, g, h) as follows:

- Map the message M to an element m of G using a reversible mapping function (appendix 4 -> add sth for 'reversible mapping function').
- Choose an integer y randomly from $\{1, \dots, q-1\}$
- Compute $s := h^y$. This is called the shared secret.
- Compute $c_1 := g^y$
- Compute $c_2 := m \cdot s$
- Bob sends the ciphertext (c_1, c_2) to Alice

Note that if one knows both the ciphertext c_1, c_2 and the plaintext m , one can easily find the shared secret s , since $c_2 \cdot m^{-1} = s$. Therefore, a new y and hence a new s is generated for every message to improve security. For this reason, y is called an ephemeral key.

Decryption Alice decrypts a ciphertext (c_1, c_2) with her private key x as follows:

- Compute $s := c_1^x$. Since $c_1 = g^y$, $c_1^x = g^{xy} = h^y$, and thus it is the same shared secret that was used by Bob in encryption.
- Compute s^{-1} , the inverse of s in the group G . This can be computed in one of several ways. If G is a subgroup of a multiplicative group of integers modulo n , where n is prime, the modular multiplicative inverse can be computed using the extended Euclidean algorithm (glossary entry). An alternative is to compute s^{-1} as c_1^{q-x} . This is the inverse of s because of Lagrange's theorem, since $s \cdot c_1^{q-x} = g^{xy} \cdot g^{(q-x)y} = (g^q)^y = e^y = e$.
- Compute $m := c_2 \cdot s^{-1}$. This calculation produces the original message m , because $c_2 = m \cdot s$; hence $c_2 \cdot s^{-1} = (m \cdot s) \cdot s^{-1} = m \cdot e = m$
- Map m back to the plaintext message M . (I still have to say above which 'reversible mapping function' I am going to choose here.

3.2.2 Additive ElGamal

Standard ElGamal encryption is homomorphic with respect to multiplication of plaintexts. That is, given two messages m_1 and m_2 , their encryptions satisfy

$$E(m_1) \cdot E(m_2) = E(m_1 \cdot m_2).$$

For our purposes, however, we require an encryption scheme that is homomorphic with respect to addition. This has to do with the Zero Knowledge Proofs mentioned later which need this additive homomorphism. Concretely, we want

$$E(m_1) \cdot E(m_2) = E(m_1 + m_2).$$

This can be achieved by a simple but important modification of ElGamal known as additive or exponential ElGamal. Instead of encrypting the message m directly, we encode the message in the exponent and encrypt g^m , where g is a generator of the underlying cyclic group (typically the same generator used to derive the public key).

Let $G = \langle g \rangle$ be a cyclic group of order q , and let the ElGamal public key be $h = g^x$, where x is the secret key. To encrypt a message $m \in \mathbb{Z}_q$, a random value $r \in \mathbb{Z}_q$ is chosen and the ciphertext is computed as

$$\text{Enc}_h(m) = (c_1, c_2) = (g^r, g^m h^r).$$

Now consider two messages m_1 and m_2 with encryptions

$$\text{Enc}_h(m_1) = (g^{r_1}, g^{m_1} h^{r_1}) \quad \text{and} \quad \text{Enc}_h(m_2) = (g^{r_2}, g^{m_2} h^{r_2}).$$

Multiplying the ciphertexts component-wise yields

$$(c_1^{(1)} c_1^{(2)}, c_2^{(1)} c_2^{(2)}) = (g^{r_1+r_2}, g^{m_1+m_2} h^{r_1+r_2}),$$

which is exactly an encryption of $m_1 + m_2$ using randomness $r_1 + r_2$. Hence,

$$\text{Enc}_h(m_1) \cdot \text{Enc}_h(m_2) = \text{Enc}_h(m_1 + m_2).$$

In this way, additive ElGamal is multiplicatively homomorphic in the ciphertext space, and this multiplicative homomorphism induces an additive homomorphism over the plaintexts when messages are encoded in the exponent. Importantly, this operation can be performed without access to the secret key.

A consequence of this construction is that ElGamal decryption yields the group element g^m rather than the plaintext value m itself. Recovering m from g^m therefore requires solving a discrete logarithm problem. In general, computing discrete logarithms in large cyclic groups is believed to be computationally infeasible, which provides the security foundation of ElGamal encryption. However, this also implies that plaintext recovery is only practical when the message space is suitably restricted.

Remarks. Since decryption involves computing a discrete logarithm, plaintexts must be chosen from a bounded domain. If m is known to lie within a small range, the discrete logarithm can be recovered via brute force by computing $g^0, g^1, g^2, \dots$ until a match with the decrypted group element is found.

For example, if plaintexts represent transaction values bounded by a maximum amount M , the receiver can recover m by evaluating g^k for all $k \in \{0, \dots, M\}$ and comparing the result to the decrypted value. As long as M is sufficiently small, this procedure is efficient in practice. Without such a bound, plaintext recovery would require solving the general discrete logarithm problem, which is computationally infeasible for appropriately chosen groups.

3.2.3 Security

Discrete Logarithm Hardness Let G be a cyclic group of prime order generated by an element g . The discrete logarithm problem is the task of recovering an exponent x from a group element g^x . No efficient algorithm is known for solving this problem in suitably chosen groups, and this hardness forms the foundation of the security of ElGamal encryption.

To illustrate this, consider the multiplicative group $\mathbb{Z}_{23}^*$ with generator $g = 5$. Suppose a transaction value is encoded as an exponent $x = 7$, resulting in the group element

$$g^x = 5^7 \bmod 23 = 17.$$

An observer of the encrypted ledger may see the value 17, but without knowledge of x cannot efficiently recover the original exponent.

Recovering x requires solving the equation

$$5^x \equiv 17 \pmod{23},$$

which entails computing the discrete logarithm of 17 with respect to base 5. While this problem can be solved by exhaustive search in small groups such as $\mathbb{Z}_{23}^*$, no efficient algorithm is known for large groups used in practice. In realistic parameter settings, the best known attacks require time exponential in the size of the group.

In the ledger setting, encrypted transaction values are represented as group elements of this form. Even though these elements encode concrete numerical amounts, extracting the underlying values would require solving discrete logarithms, which is assumed to be computationally infeasible.

For a practical implementation, the group $\mathbb{Z}_{23}^*$ is used only for illustration. In a real deployment, the cyclic group must be large enough to make exhaustive search for discrete logarithms computationally infeasible.

A natural choice satisfying the characterization above is the multiplicative group $\mathbb{Z}_p^*$, where p is a large odd prime. Such a group is cyclic, as it falls under the case $n = p^k$ with $k = 1$, and admits a generator g of full order $p - 1$.

In practice, p is chosen to be a prime of several thousand bits (e.g., $p \geq 2048$ bits), ensuring that the group order is sufficiently large to resist exhaustive search and all known generic discrete logarithm attacks. Under these parameters, recovering an exponent x from a group element $g^x \bmod p$ is believed to require time exponential in the bit length of p .

This group is particularly well suited for ElGamal encryption, as it supports efficient group operations, admits straightforward homomorphic combination of ciphertexts, and relies on well-established hardness assumptions. For these reasons, $\mathbb{Z}_p^*$ is the group used in the implementation. [2]

Final note In the above design earlier we spoke of 2 ciphertexts. This makes sense to explain the high level design of the system. In practice by using the the same randomness for both encryptions we can however encrypt the values like so

$$(g^r, g^m \cdot h_{\text{Alice}}^r, g^m \cdot h_{\text{Bob}}^r)$$

Instead of if we just ran ElGamal on both tuple entries like this:

$$(g^{r_1}, g^m \cdot h_{\text{Alice}}^{r_1}), (g^{r_2}, g^m \cdot h_{\text{Bob}}^{r_2})$$

This we can do by choosing the same randomness r for both encryptions and does not introduce any security vulnerabilities and enables a nice way to do a needed ZKP whose necessity and implementation is explained in the following sections.

3.3 Motivation for Zero Knowledge Proof concerning equality of tuple entries

This design naturally raises the question of why each transferred value is encrypted twice. If a value were encrypted only under one transacting party's public key, the other party would be unable to decrypt it and thus could neither update nor verify their own balance. Since a participant's balance is defined as the total value received minus the total value sent (possibly including newly created tokens), encrypting each transfer under both parties' public keys allows both sender and receiver to decrypt the relevant entries, maintain correct balances, and verify that sufficient funds are available for further transactions.

At the same time, participants can only decrypt tuples in which they are directly involved. Lacking the secret keys corresponding to other rows or columns of the matrix, they learn nothing about transfers between unrelated parties, thereby preserving transaction confidentiality.

This construction, however, introduces an integrity challenge: the two encrypted values in each tuple must represent the same amount. In particular, the value Alice claims to have sent to Bob must equal the value Bob claims to have received from Alice. If this condition holds for all tuples, then global balance integrity is ensured, as balances can be computed by summing received values column-wise and subtracting sent values row-wise.

Without additional safeguards, a malicious participant could violate this consistency for example, by understating an outgoing transfer while overstating the corresponding incoming value, thereby inflating a balance without actually paying. To prevent such attacks, each tuple must be accompanied by a zero-knowledge proof that the two encrypted values encode the same plaintext. This guarantees consistency and balance integrity without revealing any information about the transferred amounts.

3.4 Implementation of this ZKP

Firstly we have the following representation for matrix cell entries from above:

$$(g^r, g^m \cdot h_{\text{Alice}}^r, g^m \cdot h_{\text{Bob}}^r)$$

If we do this:

$$\frac{g^m \cdot h_{\text{Alice}}^r}{g^m \cdot h_{\text{Bob}}^r} = \left(\frac{h_{\text{Alice}}}{h_{\text{Bob}}} \right)^r$$

then we have to prove:

$$\left\{ r : g^r = a \wedge \left(\frac{h_{\text{Alice}}}{h_{\text{Bob}}} \right)^r = b \right\}$$

In the following I will explain how the equality of the exponents can be proven

Suppose there are two elements $a = g_1^m$ and $b = g_2^m$, where g_1 is a generator of a cyclic subgroup $G \subseteq \mathbb{Z}_p^*$ of prime order q , and $g_2 \in G$ is another element of the same subgroup. The goal is to prove that a and b share the same discrete logarithm with respect to g_1 and g_2 , respectively, without revealing m .

In our setting, $g_2 = h_{\text{Alice}}/h_{\text{Bob}}$, which lies in G since both public keys are powers of the same generator g .

Now one can use a standard honest-verifier zero-knowledge proof of knowledge to show that there exists an $m \in \mathbb{Z}_q$ such that

$$\log_{g_1} a = \log_{g_2} b,$$

that is, a and b contain the same discrete logarithm with respect to g_1 and g_2 , respectively, without revealing any information about m . (This proves that the m 's match, to make it a

bit more intuitive I will put below also an example when there are not same message/value encrypted.)

The protocol between a prover P and a verifier V , on common input (p, q, g_1, g_2, a, b) , proceeds as follows, where m is known only to P :

- **Commitment (P):** Choose $r \xleftarrow{\$} \mathbb{Z}_q$ uniformly at random and send

$$(k, k') = (g_1^r, g_2^r)$$

to the verifier V .

- **Challenge (V):** Choose a random challenge $c \xleftarrow{\$} \mathbb{Z}_q$ and send c to P .

- **Response (P):** Compute

$$z = r + m \cdot c \pmod{q}$$

and send z to V .

- **Verification (V):** Accept if and only if

$$g_1^z = k \cdot a^c \quad \text{and} \quad g_2^z = k' \cdot b^c$$

both hold.

Example 1: Messages do not match Suppose $a = g_1^{m_1}$ and $b = g_2^{m_2}$ with $m_1 \neq m_2$. Then the prover cannot correctly compute a response z such that both verification equations hold:

$$g_1^z = k \cdot a^c \quad \text{and} \quad g_2^z = k' \cdot b^c.$$

Even if P tries to cheat, there is no single z that satisfies both equations simultaneously, because $\log_{g_1} a \neq \log_{g_2} b$. The verifier V will detect the mismatch with overwhelming probability, and the proof fails. This shows that the protocol is sound: only matching messages pass the check.

Example 2: Messages match Now suppose $a = g_1^m$ and $b = g_2^m$ with the same m . The prover chooses a random r and sends $k = g_1^r, k' = g_2^r$ to the verifier. After receiving the challenge c , the prover computes $z = r + m \cdot c$ and sends it. The verifier checks:

$$g_1^z = g_1^{r+mc} = g_1^r \cdot g_1^{mc} = k \cdot a^c$$

$$g_2^z = g_2^{r+mc} = g_2^r \cdot g_2^{mc} = k' \cdot b^c$$

Both equalities hold perfectly because the discrete logarithms match. The verifier is convinced that the prover knows a secret m such that a and b encode the same value, but learns nothing about m itself. This demonstrates zero-knowledge: correctness is proven without revealing the secret.

Numeric Example Let $p = 23, q = 11$ (a small prime-order subgroup), and let $g_1 = 2, g_2 = 3$ in the subgroup of order q . Suppose the secret message is $m = 4$ and the prover chooses random $r = 5$.

Compute commitments:

$$k = g_1^r \pmod{p} = 2^5 \pmod{23} = 9, \quad k' = g_2^r \pmod{p} = 3^5 \pmod{23} = 10$$

Let the verifier send challenge $c = 7$. The prover computes:

$$z = r + m \cdot c \mod q = 5 + 4 \cdot 7 \mod 11 = 5 + 28 \mod 11 = 33 \mod 11 = 0$$

Verifier checks:

$$g_1^z \mod p = 2^0 \mod 23 = 1, \quad k \cdot a^c \mod p = 9 \cdot 2^{4 \cdot 7} \mod 23$$

$$2^{28} \mod 23 = 12 \Rightarrow k \cdot a^c \mod 23 = 9 \cdot 12 \mod 23 = 108 \mod 23 = 16$$

Similarly for g_2^z and $k' \cdot b^c$.

Note: With small numbers, modular arithmetic wraps around, but the example illustrates the procedure: only when a and b share the same discrete log can the verification pass. For actual cryptographic security, large primes $p \sim 2048$ bits and $q \sim 256$ bits are used.

3.5 Making the ZKP non-interactive using Fiat-Shamir Heuristics

We can see in the above example how the Zero-knowledge proof is usually interactive, this however is not all too convenient for our purposes, since we would prefer to be able to just share a Ledger and not send them back and forth a bunch of times. Then the other party simply has to accept and merge the received ledger or reject it. It is good that there is a way to make it non interactive so only the sender has to prove something and the verifier has to verify.

Explanation of Fiat Shamir Heuristics For the algorithm specified below, readers should be familiar with the multiplicative groups $\mathbb{Z}_q^*$, where q is a prime number, and Euler's totient theorem on the Euler's totient function φ .

Non-interactive Fiat-Shamir heuristic allows to replace the interactive step 3 in the above explaine ZKP with a non-interactive 'random oracle' access. In practice, we can use a cryptographic hash function instead.

- Alice wants to prove that she knows x such that $y \equiv g^x$ without revealing x .
- Alice picks a random $v \in \mathbb{Z}_q^*$ and computes $t = g^v$.
- Alice computes $c = H(g, y, t)$, where H is a cryptographic hash function.
- Alice computes $r = v - cx \mod \varphi(q)$. The resulting proof is the pair (t, r) .
- Anyone (including Bob!) can use this proof to calculate c and check whether $t \equiv g^r y^c$.

If the hash value used below does not depend on the (public) value of y , the security of the scheme is weakened, as a malicious prover can then select a certain value t so that the product cx is known.

Example (Fiat-Shamir proof works) Let $p = 23$ and let $G \subseteq \mathbb{Z}_p^*$ be the subgroup of prime order $q = 11$. Let $g = 2$ be a generator of G . Assume Alice knows a secret $x = 4$, and the public value is

$$y = g^x \mod p = 2^4 \mod 23 = 16.$$

Alice wants to prove knowledge of x non-interactively.

- Alice samples a random $v = 7 \in \mathbb{Z}_q$ and computes

$$t = g^v \mod p = 2^7 \mod 23 = 13.$$

- Alice computes the challenge using a hash function:

$$c = H(g, y, t) \in \mathbb{Z}_q.$$

For illustration, assume $c = 3$.

- Alice computes the response

$$r = v - cx \bmod q = 7 - 3 \cdot 4 \bmod 11 = -5 \bmod 11 = 6.$$

- Alice publishes the proof $(t, r) = (13, 6)$.

Anyone can now verify the proof by recomputing

$$g^r y^c \bmod p = 2^6 \cdot 16^3 \bmod 23.$$

Since

$$2^6 \bmod 23 = 18, \quad 16^3 \bmod 23 = 2,$$

we obtain

$$g^r y^c \bmod 23 = 18 \cdot 2 \bmod 23 = 36 \bmod 23 = 13 = t.$$

Thus the verifier accepts the proof. Alice has convinced the verifier that she knows x such that $y = g^x$, without revealing any information about x .

Example (Fiat–Shamir proof does not work) Now suppose Alice does not know a value x such that $y = g^x$. For example, let

$$y = 5 \in G,$$

but Alice does not know $\log_g 5$.

Alice may still attempt to construct a fake proof (t, r) . However, once the hash challenge

$$c = H(g, y, t)$$

is fixed, the verification equation

$$t \stackrel{?}{=} g^r y^c$$

implicitly enforces

$$\log_g t = r + c \cdot \log_g y.$$

Since Alice does not know $\log_g y$, she cannot ensure that this equality holds. Any choice of (t, r) will satisfy the verification equation only with negligible probability. Consequently, the verifier rejects the proof.

This illustrates the soundness of the Fiat–Shamir transformation: a valid proof can only be generated if the prover actually knows the discrete logarithm x .

Remark The examples above use small parameters for clarity. In practice, p and q are chosen to be cryptographically large, and the hash function H is modeled as a random oracle.

Application to our scheme Recall that each matrix entry in the ledger, after homomorphically adding up the set, is represented as the triple

$$(g^r, g^m h_{\text{Alice}}^r, g^m h_{\text{Bob}}^r),$$

where m is the encrypted message and r is fresh randomness.

By dividing the second and third components, the message term cancels:

$$\frac{g^m h_{\text{Alice}}^r}{g^m h_{\text{Bob}}^r} = \left(\frac{h_{\text{Alice}}}{h_{\text{Bob}}} \right)^r.$$

Let

$$g_1 = g, \quad g_2 = \frac{h_{\text{Alice}}}{h_{\text{Bob}}}, \quad a = g^r, \quad b = \left(\frac{h_{\text{Alice}}}{h_{\text{Bob}}} \right)^r.$$

Then proving that the same exponent r was used in both components is equivalent to proving

$$\log_{g_1} a = \log_{g_2} b,$$

which is exactly the statement proven by the Schnorr equality-of-discrete-logarithms protocol described above.

Using the Fiat–Shamir heuristic, this proof can be made non-interactive and attached directly to the ledger entry.

Non-interactive proof attached to a ledger entry To generate the proof, the prover (ledger creator) proceeds as follows:

- Sample $v \xleftarrow{\$} \mathbb{Z}_q$ and compute

$$t_1 = g_1^v, \quad t_2 = g_2^v.$$

- Compute the challenge

$$c = H(g_1, g_2, a, b, t_1, t_2).$$

- Compute the response

$$z = v + cr \pmod{q}.$$

- Publish the proof (t_1, t_2, z) together with the ciphertext triple.

Anyone verifying the ledger recomputes c and checks:

$$g_1^z = t_1 \cdot a^c \quad \text{and} \quad g_2^z = t_2 \cdot b^c.$$

Interpretation If the encrypted messages for Alice and Bob are the same, then the value r used in all components is consistent, and the proof verifies successfully. If the messages differ, then no single exponent r satisfies both equations, and the proof fails with overwhelming probability.

Thus, each ledger entry carries a non-interactive zero-knowledge proof that the two ciphertext components encrypt the same message, without revealing either the message m or the randomness r .

4 Code Implementation

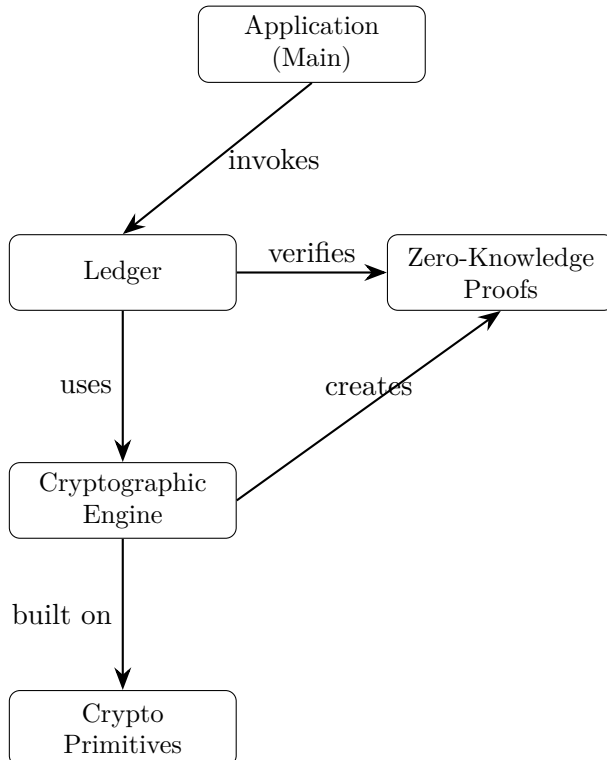
4.1 Programming Language and Design Rationale

Choice of Programming Language: The software was implemented in Java. This choice was guided by several factors: Java provides automatic memory management, which reduces the risk of memory leaks and pointer-related errors that can be challenging in C++. Additionally, Java's rich standard library and mature ecosystem facilitate rapid development and maintainability. While C++ could offer lower-level control and potentially faster execution, managing memory manually in a complex cryptographic application posed a higher risk of subtle bugs. Java thus strikes a practical balance between performance, safety, and development efficiency.

High-Level Software Design: The architecture was designed to clearly separate core responsibilities, as illustrated in the class diagram below. The design emphasizes a hierarchical and modular structure:

- **Application Layer (Main):** Entry point that orchestrates the overall flow of the program.
- **Ledger:** Handles the main transaction and state management logic.
- **Cryptographic Engine:** Provides cryptographic operations and abstractions, built upon fundamental Crypto Primitives.
- **Zero-Knowledge Proofs (ZKP):** Responsible for verification and proof generation, interacting with both the Ledger and the Cryptographic Engine.

4.2 High level design of the Software



4.3 Important Code Snippets

This section presents selected excerpts that illustrate how the high-level design is realized in code, focusing on state management, cryptographic abstraction, and zero-knowledge verification.

Ledger Representation (Ledger.java) The Ledger class maintains the global application state as a matrix of encrypted entries. Transactions are accepted only if the accompanying zero-knowledge proof verifies successfully, ensuring correctness without revealing plaintext values.

```
import java.math.BigInteger;

public class Ledger {

    private final Crypto crypto;
    private final Ciphertext[][] matrix;

    public Ledger(Crypto crypto, int size) {
        this.crypto = crypto;
        this.matrix = new Ciphertext[size][size];
    }

    public boolean submitTransaction(
        int sender,
        int receiver,
        Ciphertext amount,
        ZKP proof) {

        if (!proof.verify()) {
            return false;
        }

        matrix[sender][receiver] = amount;
        return true;
    }

    public Ciphertext getEntry(int i, int j) {
        return matrix[i][j];
    }
}
```

Cryptographic Group Abstraction (CryptoGroup.java) The CryptoGroup class encapsulates low-level group operations and parameters. Centralizing these primitives improves clarity, correctness, and modularity of the cryptographic code.

```
import java.math.BigInteger;

public class CryptoGroup {

    public final BigInteger p; // modulus
    public final BigInteger q; // group order
    public final BigInteger g; // generator
```

```

public CryptoGroup(BigInteger p, BigInteger q, BigInteger g) {
    this.p = p;
    this.q = q;
    this.g = g;
}

public BigInteger mul(BigInteger a, BigInteger b) {
    return a.multiply(b).mod(p);
}

public BigInteger pow(BigInteger base, BigInteger exp) {
    return base.modPow(exp, p);
}

public BigInteger inverse(BigInteger a) {
    return a.modInverse(p);
}
}

```

Encrypted Equality Zero-Knowledge Proof (EqualityZKP.java) The EqualityZKP class implements a non-interactive zero-knowledge proof that two ciphertexts encrypt the same value. Verification relies solely on public data, preserving privacy while enabling correctness checks.

```

import java.math.BigInteger;

public class EqualityZKP implements ZKP {

    public final BigInteger k1; //  $g_1^r$ 
    public final BigInteger k2; //  $g_2^r$ 
    public final BigInteger z; // response

    private final CryptoGroup group;
    private final BigInteger g1;
    private final BigInteger g2;
    private final BigInteger a;
    private final BigInteger b;

    public EqualityZKP(
        CryptoGroup group,
        BigInteger g1,
        BigInteger g2,
        BigInteger a,
        BigInteger b,
        BigInteger k1,
        BigInteger k2,
        BigInteger z) {

        this.group = group;
        this.g1 = g1;
        this.g2 = g2;
    }
}

```

```

        this.a = a;
        this.b = b;
        this.k1 = k1;
        this.k2 = k2;
        this.z = z;
    }

    @Override
    public boolean verify() {
        BigInteger c = FiatShamir.hashToZq(group.q, g1, g2, a, b, k1, k2);

        BigInteger lhs1 = group.pow(g1, z);
        BigInteger rhs1 = group.mul(k1, group.pow(a, c));

        BigInteger lhs2 = group.pow(g2, z);
        BigInteger rhs2 = group.mul(k2, group.pow(b, c));

        return lhs1.equals(rhs1) && lhs2.equals(rhs2);
    }
}

```

5 Evaluation

This section documents practical observations from the prototype implementation. The intent is not a full performance benchmark, but to summarize what the implementation does, what it costs in runtime and storage, and what scale it can plausibly handle.

5.1 Implementation Setup

The prototype is implemented in Java and models encrypted ledger state, transaction submission, and proof verification. The ledger is a fixed $n \times n$ matrix of ElGamal ciphertexts indexed by sender/receiver. Each submitted transaction includes a ciphertext and a zero-knowledge proof object; the proof is verified at submission time.

The cryptographic group uses the RFC 3526 2048-bit MODP prime (group 14) with $q = (p-1)/2$ and generator $g = 4$. This group is widely used in practice and is commonly associated with ≈ 128 -bit security for discrete-log-based protocols.[3] The code validates primality of p and q and checks that g has order q at startup.

The implementation currently includes:

- Additive (exponential) ElGamal encryption and homomorphic combination of ciphertexts.
- A Schnorr-style equality-of-discrete-log ZKP using Fiat–Shamir.
- A placeholder non-negative balance proof interface (verification always accepts; no real range proof is implemented yet).

The system is single-process and does not implement a replication or merge protocol; it focuses on the cryptographic data structures and verification logic.

5.2 Runtime Characteristics

The dominant cost per transaction is modular exponentiation. An encryption performs a small number of exponentiations and multiplications, and equality ZKP verification requires a con-

stant number of exponentiations plus a hash. In practice, verification is the main compute cost per submitted transaction.

Because the implementation performs only a handful of exponentiations per transaction, interactive use is feasible for small to moderate workloads. No batching or precomputation is implemented, so the cost scales linearly with the number of transactions verified.

5.3 Storage Requirements

The ledger stores one ciphertext per matrix cell and does not retain proofs after verification. Consequently, storage grows as $O(n^2)$ ciphertexts for n participants, regardless of how many transactions occur per pair. This keeps memory use bounded but means transaction history per pair is overwritten, not accumulated.

Each ciphertext consists of two group elements, which is significantly larger than a plaintext balance update. The overhead is expected for confidential ledgers with public verification.

5.4 Scaling Considerations

To align with the feedback, consider $n = 100$ participants and $t = 200$ transactions per participant. In the current prototype:

- **Runtime.** There are $n \cdot t = 20,000$ transaction submissions to verify. Since each verification is constant-time in group operations, total runtime scales linearly with 20k proofs. This is feasible for a single-machine prototype, but verification remains the dominant cost.
- **Storage.** The ledger stores $n^2 = 10,000$ ciphertexts. Because entries overwrite, storage is independent of t in the current code. A history-preserving design would instead grow proportional to $n \cdot t$ and store proofs, significantly increasing space usage.

Overall, the prototype can handle small to medium group sizes with moderate transaction rates. Two limitations affect scalability and security at larger scales: (i) the non-negative balance proof is only a stub and would need to be replaced with a real range proof, which would increase computation and storage; and (ii) the current ledger structure does not retain history, limiting auditability.

A natural extension (discussed later) is incremental or batched proof verification to reduce per-transaction overhead when many proofs are validated in sequence.

6 Possible Extensions

The construction presented so far focuses on confidentiality and mergeability of ledger state under encryption. However, fully preventing double spending in this setting requires an additional mechanism that is not implemented in the current system. We briefly outline such a mechanism here as a possible extension and leave its full realization to future work.

6.1 Second Zero-Knowledge Proof for Balance Preservation

To prevent double spending, a second zero-knowledge proof is required that allows a receiver to verify that a sender is not spending more funds than they possess, without revealing the sender’s actual balance.

In the proposed ledger representation, the balance of an account is defined as the difference between the total value received and the total value spent. Concretely, this corresponds to the sum of the encrypted values in the account’s column of the transaction matrix minus the sum of the encrypted values in the corresponding row. Intuitively, column entries represent incoming funds, while row entries represent outgoing payments.

For a transfer of a given amount to be valid, the sender must therefore demonstrate that their current balance is greater than or equal to the value being transferred. This condition must be verified without revealing either the balance itself or any intermediate sums.

Such a check can be implemented using the homomorphic properties of the encryption scheme. Encrypted values within each relevant cell set can be added homomorphically, and these sums can in turn be aggregated to obtain encrypted representations of the sender’s total incoming and outgoing funds. The sender can then homomorphically subtract the outgoing sum and the transaction amount from the incoming sum.

If the resulting encrypted value represents a non-negative number, the sender can produce a zero-knowledge proof that the balance constraint is satisfied. A balance of zero is permitted; only negative balances are disallowed.

An equivalent formulation is to first add the transaction amount to the appropriate cell set and then prove, in zero knowledge, that the resulting balance (column sum minus row sum) is non-negative. In both formulations, the proof guarantees that no double spending has occurred while revealing no additional information about the sender’s balance or transaction history.

Implementation Considerations A concrete realization of this proof could be based on range proofs for homomorphic encryption schemes such as ElGamal. Designing and integrating such a proof system in a sound and efficient way is outside the scope of this work and is left as a direction for future research.

7 Conclusion

This thesis investigated how confidentiality can be added to a GOC-style ledger design by encrypting transaction contributions and verifying structural consistency without revealing plaintexts. The prototype implementation demonstrates the cryptographic core: additive (exponential) ElGamal encryption in a 2048-bit MODP group and a non-interactive equality-of-discrete-log proof (Fiat–Shamir with SHA-256) to check that two ciphertexts were formed with the same randomness. This enables equality checks over encrypted values and shows that verification can be performed using public information only.

The implementation focuses on the cryptographic mechanisms and a minimal ledger data model. It stores a fixed $n \times n$ matrix of ciphertexts and verifies proofs at submission time. It does not implement replication, merging, or state-based CRDT semantics, and it does not retain proofs after verification. Likewise, the non-negative balance proof is currently a stub and does not provide a real range-proof guarantee.

Within these bounds, the prototype supports the main technical claim: privacy-preserving verification of consistency between encrypted entries is feasible with standard cryptographic tools. Integrating this approach into a full CRDT ledger remains future work and would require (i) a merge protocol, (ii) history-preserving storage, and (iii) real range proofs, together with optimizations such as batching or incremental proof verification.

Overall, the work provides a concrete starting point for combining homomorphic encryption and zero-knowledge proofs with weak-consistency ledger designs, while making clear which system-level properties are design goals rather than implemented features.

8 AI Disclaimer

I used AI-based tools to support parts of the writing and software development in this work.

For the text of the manuscript, I used conversational AI tools like ChatGPT in an iterative process to refine formulations and improve clarity. Typically, I drafted or outlined passages myself, asked the AI for suggestions or alternative phrasings, and then reviewed, edited, and

reformulated the responses. This back-and-forth process was sometimes repeated multiple times until I arrived at a formulation that I considered appropriate. All final text was reviewed and adapted by me.

For parts of the code, I used OpenAI Codex. I provided detailed prompts describing the intended architecture and high-level functionality of the system, after which Codex generated initial code suggestions. I then iteratively refined the generated code through debugging and review, requesting modifications or rewrites where necessary and manually adapting the results until the software functioned as intended.

References

- [1] Erick Lavoie et al. *GOC Ledger: A Conflict-Free Replicated Ledger*. Tech. rep. Specification document. Google, 2023.
- [2] Christof Paar, Jan Pelzl, and Tim Güneysu. *Understanding Cryptography: From Established Symmetric and Asymmetric Ciphers to Post-Quantum Algorithms*. 2nd. See Chapter 8.5 on ElGamal Encryption. Cham, Switzerland: Springer, 2024. ISBN: 978-3031578554.
- [3] Tero Kivinen and Mika Kojo. *More Modular Exponential (MODP) Diffie-Hellman Groups for Internet Key Exchange (IKE)*. Tech. rep. RFC 3526. Internet Engineering Task Force (IETF), May 2003. DOI: 10.17487/RFC3526. URL: <https://datatracker.ietf.org/doc/html/rfc3526>.
- [4] Cryptography Stack Exchange. *Additive ElGamal Encryption Algorithm*. Accessed: 2026-03-13. 2013. URL: <https://crypto.stackexchange.com/questions/9000/additive-elgamal-encryption-algorithm>.
- [5] Cryptography Stack Exchange. *Can ElGamal be Made Additively Homomorphic and How Could It Be Used for E-Voting?* Accessed: 2026-03-13. 2012. URL: <https://crypto.stackexchange.com/questions/3626/can-elgamal-be-made-additively-homomorphic-and-how-could-it-be-used-for-e-voting>.
- [6] Cryptography Stack Exchange. *Is It Possible to Create a Zero-Knowledge Proof that a Number Is More than Zero?* Accessed: 2026-03-13. 2017. URL: <https://crypto.stackexchange.com/questions/53745/is-it-possible-to-create-a-zero-knowledge-proof-that-a-number-is-more-than-zero>.
- [7] MathOverflow. *Zero-Knowledge Proof of Equality*. Accessed: 2026-03-13. 2016. URL: <https://mathoverflow.net/questions/236473/zero-knowledge-proof-of-equality>.
- [8] Alfred J. Menezes, Paul C. van Oorschot, and Scott A. Vanstone. *Handbook of Applied Cryptography*. Chapter 8: Public-Key Encryption. CRC Press, 1996. URL: <https://cacr.uwaterloo.ca/hac/about/chap8.pdf>.
- [9] Wikipedia contributors. *Cyclic Group*. Accessed: 2026-03-13. 2026. URL: https://en.wikipedia.org/wiki/Cyclic_group.
- [10] Wikipedia contributors. *Multiplicative Group of Integers Modulo n* . Accessed: 2026-03-13. 2026. URL: https://en.wikipedia.org/wiki/Multiplicative_group_of_integers_modulo_n.
- [11] Wikipedia contributors. *Decisional Diffie-Hellman Assumption*. Accessed: 2026-03-13. 2026. URL: https://en.wikipedia.org/wiki/Decisional_Diffie%E2%80%93Hellman_assumption.
- [12] David Chaum and Torben P. Pedersen. “Wallet Databases with Observers”. In: *Advances in Cryptology — CRYPTO ’92*. Vol. 740. Lecture Notes in Computer Science. Springer, 1993, pp. 89–105. DOI: 10.1007/3-540-48071-4_7.